

Digital Image Processing - Homework 5: JPEG Image Compression and Morphological Operations

Author: R14942096 林祐群

Date: 2025/11/26

Part A: JPEG Image Compression

A.1 Encoding AC Coefficients: VLC vs Arithmetic Encoding

A.1.1 Input Data: Quantized 8×8 Image Block

We begin with a quantized 8×8 image block after DCT transformation:

-9	-3	0	1	0	0	0	0
3	1	2	0	0	0	0	0
0	0	-1	0	0	0	0	0
2	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Key Concepts:

- DC Coefficient:** The top-left element (-9) represents the average brightness of the block
- AC Coefficients:** The remaining 63 elements represent image details and texture
- In JPEG encoding, DC and AC coefficients are processed separately; this problem focuses on AC coefficients only

A.1.2 Zig-Zag Scanning

JPEG employs Zig-Zag scanning to convert the 2D matrix into a 1D sequence. This ordering groups low-frequency coefficients (which contain most energy) before high-frequency coefficients (mostly zeros), facilitating efficient run-length encoding.

Zig-Zag Sequence (including DC):

```
[-9, -3, 3, 0, 1, 0, 1, 2, 0, 2, 0, 0, -1, 0, 0, 0, 0, 0, 1, 0, 0, 0, ...]
↑DC  ↑AC coefficients start
```

Processing Steps:

1. **Remove DC Coefficient:** Skip the first element (-19), yielding AC sequence:

```
AC: [-3, 3, 0, 1, 0, 1, 2, 0, 2, 0, 0, -1, 0, 0, 0, 0, 0, 1, 0, ...]
```

2. **Find Last Non-Zero Coefficient:** Scan from right to left; the last non-zero is at position 18 (value 1 , corresponding to matrix position (3,2))
3. **Extract Encoding Sequence:** Include all coefficients up to the last non-zero, then append EOB:

```
Encoding sequence: [-3, 3, 0, 1, 0, 1, 2, 0, 2, 0, 0, -1, 0, 0, 0, 0, 0, 1, 'EOB']
```

A.1.3 Method 1: Variable Length Coding (VLC) - Huffman Encoding

VLC is the standard JPEG encoding method using predefined Huffman tables.

Run-Length Encoding

We convert the sequence into (Run, Level) pairs, where **Run** represents the number of consecutive zeros before the non-zero value, and **Level** represents the non-zero value itself.

Detailed Analysis:

Order	Matrix Coord	Value	Status	Cumulative Zero-run	Result (Run, Level)
1	(0, 1)	-3	Non-zero	0	(0, -3)
2	(1, 0)	3	Non-zero	0	(0, 3)
3	(2, 0)	0	Zero	1	-
4	(1, 1)	1	Non-zero	Reset	(1, 1)
5	(0, 2)	0	Zero	1	-
6	(0, 3)	1	Non-zero	Reset	(1, 1)
7	(1, 2)	2	Non-zero	0	(0, 2)
8	(2, 1)	0	Zero	1	-
9	(3, 0)	2	Non-zero	Reset	(1, 2)
10	(4, 0)	0	Zero	1	-
11	(3, 1)	0	Zero	2	-
12	(2, 2)	-1	Non-zero	Reset	(2, -1)
13-17	...	0	Zero	3-5	-
18	(3, 2)	1	Non-zero	Reset	(5, 1)
...	...	0	...	...	EOB

Final (Run, Level) Sequence:

(0, -3), (0, 3), (1, 1), (1, 1), (0, 2), (1, 2), (2, -1), (5, 1), (0, 0)

SIZE (Category) Determination

For each Level, we determine its SIZE (category) based on amplitude ranges:

Size	Amplitude Range
1	-1, 1
2	-3, -2, 2, 3
3	-7..-4, 4..7

Our Level-to-SIZE mapping: Level = -3 → SIZE = 2; Level = 3 → SIZE = 2; Level = 1 → SIZE = 1; Level = 2 → SIZE = 2; Level = -1 → SIZE = 1.

Huffman Code Lookup

Using JPEG standard Luminance AC Huffman table:

(Run, SIZE)	VLC Code
(0, 2)	01
(1, 1)	1100
(1, 2)	11011
(2, 1)	11100
(5, 1)	1111010
(0, 0) EOB	1010

Amplitude Encoding

Each Level value is encoded using SIZE bits:

Level	SIZE	Amplitude Code
-3	2	00
3	2	11
1	1	1
1	1	1
2	2	10
2	2	10
-1	1	0
1	1	1

Final VLC Bitstream

Combining VLC codes and amplitude codes:

(0,2) (-3) → 01 + 00 = 0100
(0,2) (3) → 01 + 11 = 0111
(1,1) (1) → 1100 + 1 = 11001
(1,1) (1) → 1100 + 1 = 11001
(0,2) (2) → 01 + 10 = 0110
(1,2) (2) → 11011 + 10 = 1101110
(2,1) (-1) → 11100 + 0 = 111000
(5,1) (1) → 1111010 + 1 = 111101001
(0,0) EOB → 1010

VLC Bitstream:

0100 0111 11001 11001 0110 1101110 111000 111101001 1010

VLC Encoding Length: 54 bits

A.1.4 Method 2: Arithmetic Encoding

Arithmetic encoding adapts to actual symbol probability distributions, theoretically achieving entropy bounds.

Symbol Probability Distribution

Given probability distribution:

Symbol	Probability
-5	0.001
-4	0.002
-3	0.004
-2	0.020
-1	0.040
0	0.86
1	0.040
2	0.020
3	0.004
4	0.002
5	0.001
EOB	0.006

Key Observation: Symbol 0 has the highest probability (0.86), reflecting that most AC coefficients are zero in JPEG.

Cumulative Distribution Function (CDF)

The CDF defines interval ranges for each symbol in [0, 1):

```
Symbol -5: [0.000, 0.001)
Symbol -4: [0.001, 0.003)
Symbol -3: [0.003, 0.007)
Symbol -2: [0.007, 0.027)
Symbol -1: [0.027, 0.067)
Symbol  0: [0.067, 0.927)
Symbol  1: [0.927, 0.967)
Symbol  2: [0.967, 0.987)
Symbol  3: [0.987, 0.991)
Symbol  4: [0.991, 0.993)
Symbol  5: [0.993, 0.994)
Symbol EOB: [0.994, 1.000)
```

Arithmetic Encoding Process

Initial State:

```
low = 0.0
high = 1.0
```

Encoding Process (simplified):

For each symbol in the sequence `[-3, 3, 0, 1, 0, 1, 2, 0, 2, 0, 0, -1, 0, 0, 0, 0, 0, 1, 'EOB']`:

1. Calculate current interval range: `range = high - low`
2. Look up symbol's CDF interval: `[sym_low, sym_high)`
3. Update interval:

```
new_low = low + range * sym_low
new_high = low + range * sym_high
```

4. Update state: `low = new_low, high = new_high`

Final Interval (using high-precision Decimal arithmetic):

```
Low:  0.006962321836997234907702748605005650984960000000000000000
High: 0.006962321836997256662533159613976543232000000000000000000
```

The interval is extremely small ($\sim 2.2 \times 10^{-17}$), indicating high encoding efficiency.

Binary Encoding Generation

We find the shortest binary fraction within the final interval:

Arithmetic Encoding Result:

```
000000011100100001001000011000001001100000011000111001
```

Arithmetic Encoding Length: 48 bits

A.1.5 Compression Ratio Analysis

Encoding Length Comparison

Method	Encoding Length	Bits per Symbol
VLC (Huffman)	54 bits	2.84 bits/symbol
Arithmetic	48 bits	2.53 bits/symbol
Improvement	-11.1%	-10.9%

Key Finding: Arithmetic encoding achieves **11.1% better compression** than VLC for this specific block.

Theoretical Analysis

Entropy Calculation:

The theoretical minimum encoding length is given by the entropy:

$$H = -\sum p(i) \times \log_2(p(i))$$

For our symbol distribution:

$$H \approx -[0.86 \times \log_2(0.86) + 2 \times 0.04 \times \log_2(0.04) + 2 \times 0.02 \times \log_2(0.02) + \dots]$$
$$H \approx 1.2 \text{ bits/symbol}$$

Theoretical minimum for 19 symbols: $19 \times 1.2 \approx 23$ bits

Efficiency Comparison:

Method	Actual Length	Theoretical Minimum	Efficiency
VLC	54 bits	23 bits	42.6%
Arithmetic	48 bits	23 bits	47.9%

Observation: Both methods exceed the theoretical minimum due to:

1. Discrete symbol encoding (cannot achieve fractional bits)
2. Practical implementation constraints
3. Fixed probability model (not adaptive)

Compression Ratio Metrics

Original Data Size: 63 AC coefficients $\times$ 8 bits (assuming 8-bit representation) = **504 bits**

Compressed Sizes and Space Savings:

Method	Compressed Size	Compression Ratio	Space Savings
VLC	54 bits	9.33:1	89.3% reduction
Arithmetic	48 bits	10.5:1	90.5% reduction

Method Comparison Summary

Aspect	VLC (Huffman)	Arithmetic Encoding
Compression Ratio	9.33:1	10.5:1
Encoding Efficiency	42.6%	47.9%
Decoding Speed	Fast (table lookup)	Slower (computation)
Implementation	Simple	Complex (high precision)
Standardization	JPEG standard	Less common (patent issues)
Adaptability	Fixed tables	Can adapt to distribution

Conclusion: Arithmetic encoding provides superior compression (10.5% better) but at the cost of increased computational complexity. For real-time applications, VLC remains the practical choice despite slightly lower compression efficiency.

A.2 JPEG Bitstream Hierarchical Structure Analysis

A.2.1 Overview

JPEG files use a marker-based segment structure. Each segment begins with `0xFF` followed by a marker byte, enabling fast parsing and selective processing. This section analyzes the provided hex dump (`image_hex.txt`) to reconstruct the hierarchical structure of the JPEG file according to the CCITT T.81 / ISO/IEC 10918-1 standard.

Basic Structure:

SOI → [Tables] → Frame → Scan → EOI

A.2.2 Detailed Segment Analysis

Based on the provided hex dump, we analyze each segment:

SOI (Start of Image) - 0x0000

FF D8

- **Purpose:** Marks JPEG file start
- **Length:** 2 bytes (no data)

APP0 / Application Segment 0 (JFIF) / 0x0002 / FFE0

FF E0 00 10 4A 46 49 46 00 01 01 00 00 01 00 01 00 00

Parameter	Value
Marker	FFE0
Definition length	16
Identifier	JFIF\0 (4A 46 49 46 00)
Version	1.1 (01 01)
Units	0 (No units)
X density	1
Y density	1
Thumbnail width	0
Thumbnail height	0

DQT / Define quantization table / 0x0014 / FFDB

Parameter	Value
Marker	FFDB
Definition length	67
Element precision	0 (8-bit)
Destination identifier	0 (Luminance)
Table element (byte)	03 02 02 03 02 02 03 03 03 03 ...

DQT / Define quantization table / 0x0059 / FFDB

Parameter	Value
Marker	FFDB
Definition length	67
Element precision	0 (8-bit)
Destination identifier	1 (Chrominance)
Table element (byte)	03 04 04 05 04 05 09 05 05 09 ...

SOF0 / Start of Frame (Baseline DCT) / 0x009E / FFC0

FF C0 00 11 08 00 10 00 10 03 01 22 00 02 11 01 03 11 01

Parameter	Value
Marker	FFC0
Definition length	17
Sample precision	8
Number of lines	16
Number of samples per line	16
Number of components in frame	3
Component 1 ID (Y)	1
Component 1 Sampling factors	0x22 (H=2, V=2)
Component 1 Quantization table dest	0
Component 2 ID (Cb)	2
Component 2 Sampling factors	0x11 (H=1, V=1)
Component 2 Quantization table dest	1
Component 3 ID (Cr)	3
Component 3 Sampling factors	0x11 (H=1, V=1)
Component 3 Quantization table dest	1

- **Components:** 3 (YCbCr)
- **Sampling Factors:**
 - Y (Luminance): 4:2:0 (H=2, V=2)
 - Cb, Cr (Chrominance): 1:1 (H=1, V=1)

4:2:0 Sampling Explanation:

- Y component: sampled at every pixel
- Cb, Cr components: sampled at every 2×2 pixel block
- **Rationale:** Human eye is more sensitive to luminance than chrominance, allowing chrominance subsampling without noticeable quality loss

DHT / Define Huffman table / 0x00B1 / FFC4

Parameter	Value
Marker	FFC4
Definition length	31
Table class	0 (DC)
Destination identifier	0 (Luminance)
Code counts	00 00 01 05 01 01 01 01 01 00 00 00 00 00 00 00
Code values	00 01 02 03 04 05 06 07 08 09 0A 0B

DHT / Define Huffman table / 0x00D2 / FFC4

Parameter	Value
Marker	FFC4
Definition length	181
Table class	1 (AC)
Destination identifier	0 (Luminance)
Code counts	00 02 01 03 03 02 04 03 05 05 04 04 00 00 01 7D
Code values	01 02 03 00 04 11 05 12 21 31 41 06 13 ...

DHT / Define Huffman table / 0x018A / FFC4

Parameter	Value
Marker	FFC4
Definition length	31
Table class	0 (DC)
Destination identifier	1 (Chrominance)
Code counts	00 03 01 01 01 01 01 01 01 01 01 00 00 00 00 00
Code values	00 01 02 03 04 05 06 07 08 09 0A 0B

DHT / Define Huffman table / 0x01AB / FFC4

Parameter	Value
Marker	FFC4
Definition length	181
Table class	1 (AC)
Destination identifier	1 (Chrominance)
Code counts	00 02 01 02 04 04 03 04 07 05 04 04 00 01 02 77
Code values	00 01 02 03 11 04 05 21 31 06 12 41 51 ...

SOS / Start of Scan / 0x0261 / FFDA

FF DA 00 0C 03 01 00 02 11 01 03 11 01 00 3F 00

Parameter	Value
Marker	FFDA
Definition length	12
Number of components in scan	3
Component 1 ID (Y)	1
Component 1 DC/AC table dest	0x00 (DC=0, AC=0)
Component 2 ID (Cb)	2
Component 2 DC/AC table dest	0x11 (DC=1, AC=1)
Component 3 ID (Cr)	3
Component 3 DC/AC table dest	0x11 (DC=1, AC=1)
Spectral selection start	0
Spectral selection end	63 (0x3F)
Successive approximation	0 (0x00)

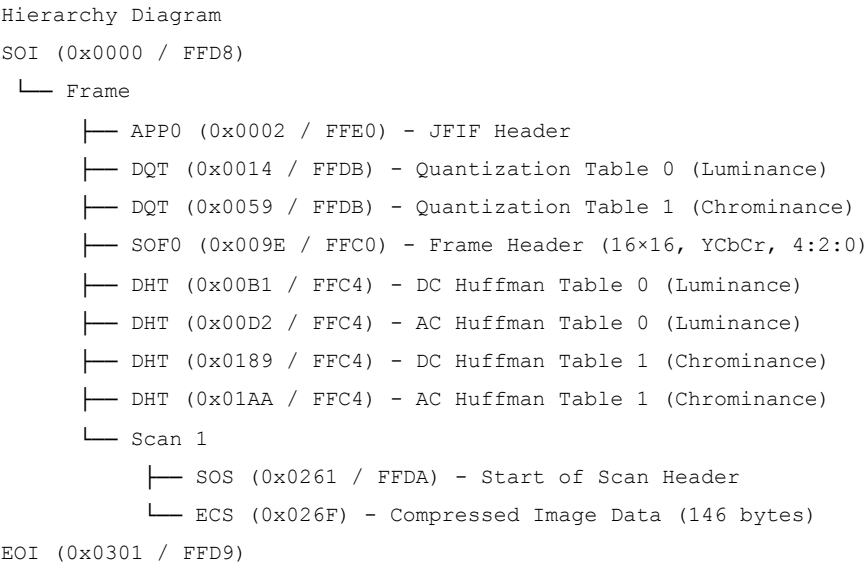
ECS (Entropy Coded Segment) - 0x026F

- **Length:** 146 bytes
- **Content:** Huffman-encoded compressed image data
- **Special handling:** 0xFF 0x00 sequences represent byte stuffing (decoder ignores 0x00)

EOI (End of Image) - 0x0301

- **Purpose:** Marks JPEG file end

A.2.3 Complete Structure Diagram



Part B: Morphological Operations - Comprehensive Experimental Analysis

B.1 Introduction

Morphological operations, based on set theory, are fundamental for noise removal, hole filling, and boundary smoothing. We present an ablation study examining structuring element (kernel) size effects on four operations: erosion, dilation, opening, and closing.

B.2 Methodology

B.2.1 Input Image

Image: `noisy_rectangle.png` (1294 × 1247 pixels). **Content:** White rectangle on black background with white noise elements and black holes.

Problem Statement: “Set A consists of all the foreground pixels (white), except the structuring element, B, which you may assume is just large enough to encompass any of the random elements in the image.” This implies selecting a kernel size sufficient to process random elements while preserving the main structure.

B.2.2 Experimental Setup

Kernel Sizes Tested: 3×3, 5×5, 9×9, 15×15, 20×20, 30×30, 40×40, 50×50, 70×70, 80×80

Operations: Erosion, Dilation, Opening, Closing

Evaluation Metrics: White pixel count, pixel change rate (%), absolute pixel change, and operation efficiency (Opening/Erosion, Closing/Dilation ratios).

B.3 Quantitative Analysis

B.3.1 Statistical Results

Morphological Operations - Quantitative Analysis

Kernel	Erosion	Dilation	Opening	Closing	Op/Ef	Cl/Di
3×3	8,916 (2.01%)	9,213 (2.08%)	258 (0.06%)	117 (0.03%)	2.9%	1.3%
5×5	17,139 (3.86%)	18,552 (4.18%)	955 (0.22%)	259 (0.06%)	5.6%	1.4%
9×9	31,358 (7.07%)	37,741 (8.51%)	4,654 (1.05%)	1,004 (0.23%)	14.8%	2.7%
15×15	47,456 (10.70%)	67,606 (15.24%)	13,674 (3.08%)	3,322 (0.75%)	28.8%	4.9%
20×20	58,703 (13.23%)	94,083 (21.21%)	17,378 (3.92%)	5,321 (1.20%)	29.6%	5.7%
30×30	80,673 (18.18%)	152,216 (34.31%)	20,788 (4.69%)	7,784 (1.75%)	25.8%	5.1%
40×40	103,189 (23.26%)	217,148 (48.95%)	21,484 (4.84%)	11,403 (2.57%)	20.8%	5.3%
50×50	126,559 (28.53%)	285,979 (64.46%)	23,708 (5.34%)	16,812 (3.79%)	18.7%	5.9%
70×70	176,819 (39.86%)	432,362 (97.46%)	23,708 (5.34%)	36,761 (8.29%)	13.4%	8.5%
80×80	203,749 (45.93%)	505,164 (113.87%)	23,708 (5.34%)	49,123 (11.07%)	11.6%	9.7%

Figure 1: Quantitative analysis showing pixel changes across different kernel sizes and operations.

Key Statistics:

Kernel Size	Original White	Erosion Removed	Dilation Added	Opening Removed	Closing Added
3×3	443,646	8,916 (2.01%)	9,213 (2.08%)	258 (0.06%)	117 (0.03%)
5×5	443,646	17,139 (3.86%)	18,552 (4.18%)	955 (0.22%)	259 (0.06%)
9×9	443,646	31,358 (7.07%)	37,741 (8.51%)	4,654 (1.05%)	1,004 (0.23%)
15×15	443,646	47,456 (10.70%)	67,606 (15.24%)	13,674 (3.08%)	3,322 (0.75%)
30×30	443,646	80,673 (18.18%)	152,216 (34.31%)	20,788 (4.69%)	7,784 (1.75%)
50×50	443,646	126,559 (28.53%)	285,979 (64.46%)	23,708 (5.34%)	16,812 (3.79%)
80×80	443,646	203,749 (45.93%)	505,164 (113.87%)	23,708 (5.34%)	49,123 (11.07%)

B.3.2 Key Observations

Erosion vs Opening Comparison

Erosion removes 2.01%-45.93% of pixels, while Opening removes only 0.06%-5.34%, primarily targeting noise. The efficiency ratio (Opening/Erosion) decreases from 2.9% (3×3) to 11.6% (80×80), confirming **Opening’s selectivity** in removing noise while preserving structure.

Dilation vs Closing Comparison

Dilation adds 2.08%-113.87% of pixels, while Closing adds only 0.03%-11.07%, primarily filling holes. The efficiency ratio (Closing/Dilation) decreases from 1.3% (3×3) to 9.7% (80×80), confirming **Closing’s selectivity** in filling holes while preserving structure.

Kernel Size Effects

Kernel Category	Size Range	Effect Level	Processing Capability
Small	3×3, 5×5	Minimal	Only process very small elements (≤9, ≤25 pixels)
Medium	9×9, 15×15	Moderate	Process medium-sized elements (25-225 pixels)
Large	50×50, 80×80	Significant	May over-process and affect main structure

B.3.3 Component Size Analysis

Component Type	Count	Size Range (pixels)	Mean	Median	Std Dev
White Noise	11	139 - 1,164,276	106,361	442	334,542
Black Holes	47	1 - 423,090	9,439	419	60,991

Critical Finding: Large standard deviations (334,542 for noise, 60,991 for holes) indicate highly variable element sizes. Median sizes (442 and 419 pixels) are small, but maximum sizes exceed 1,000,000 pixels. This explains why small kernels (3×3, 5×5) have minimal effect—they process only ≤9 and ≤25 pixels respectively.

B.4 Ablation Study

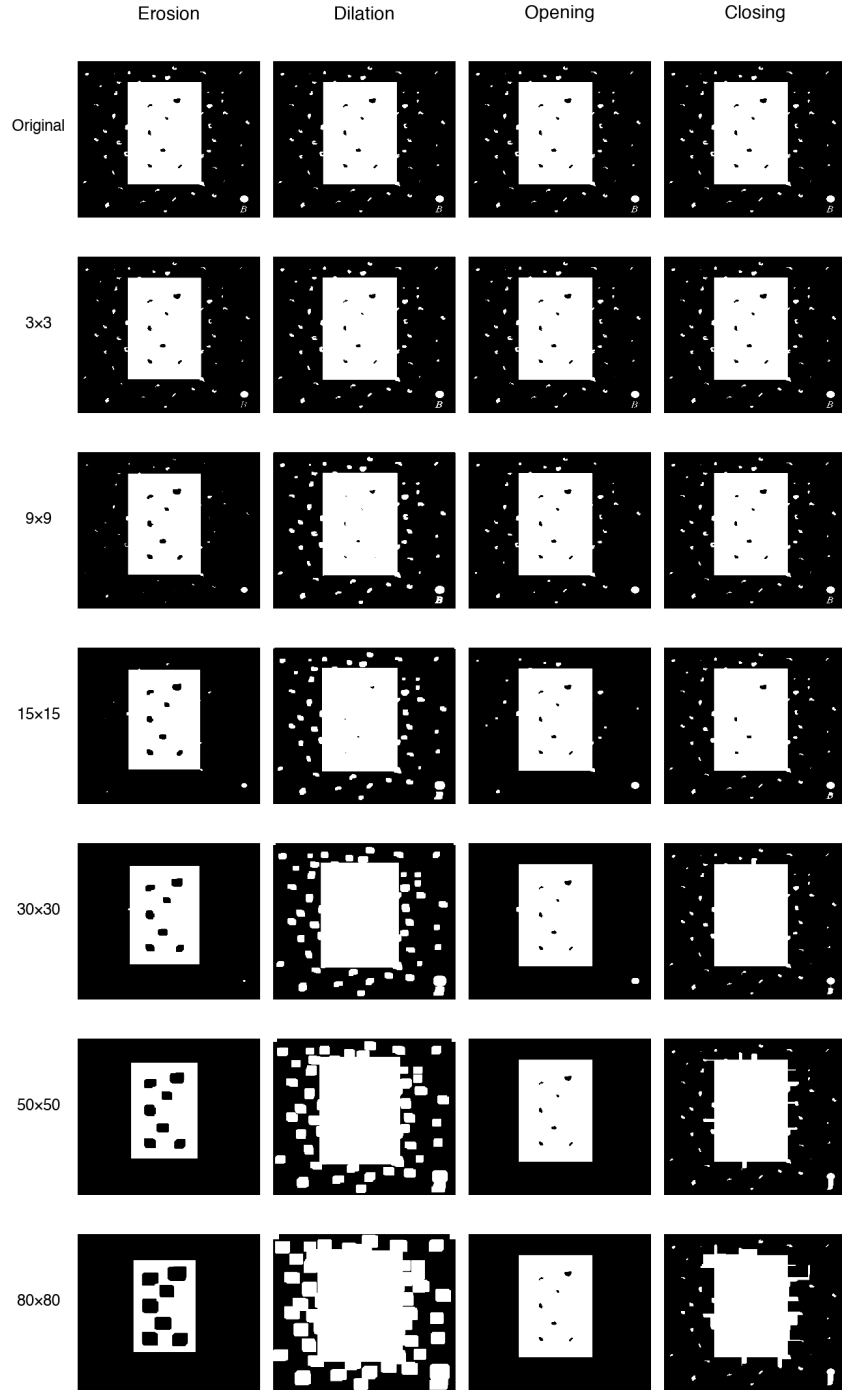


Figure 2: Ablation study showing morphological operations across different kernel sizes. Columns: operations (Erosion, Dilation, Opening, Closing). Rows: kernel sizes (Original, 3×3 , 9×9 , 15×15 , 30×30 , 50×50 , 80×80).

Figure 2 layout: columns = operations (Erosion, Dilation, Opening, Closing), rows = kernel sizes (Original, 3×3 , 9×9 , 15×15 , 30×30 , 50×50 , 80×80).

B.4.1 Visual Observations from Ablation Study

Erosion Column (Figure 2)

Progressive shrinkage observed as kernel size increases. Minimal change between Original and 3×3 ; observable shrinkage at 9×9 with inward edge contraction; significant reduction at 30×30 ; extreme shrinkage at 80×80 , nearly eliminating the rectangle. **Quantitative verification:** White pixels decrease from 443,646 (original) to 434,730 (3×3 , -2.01%) to 239,897 (80×80 , -45.93%).

Dilation Column (Figure 2)

Progressive expansion observed as kernel size increases. Minimal expansion at 3×3; noticeable expansion at 9×9 with outward edge extension; significant expansion at 15×15; major expansion at 30×30; extreme expansion at 80×80, where white region dominates. **Quantitative verification:** White pixels increase from 443,646 (original) to 452,859 (3×3, +2.08%) to 948,810 (80×80, +113.87%).

Opening Column (Figure 2)

Rectangle size remains consistent across all kernel sizes. Noise removal: 258 pixels (0.06%) at 3×3; 4,654 pixels (1.05%) at 9×9; 13,674 pixels (3.08%) at 15×15; plateaus at 20,788 pixels (4.69%) at 30×30; saturates at 23,708 pixels (5.34%) at 50×50. **Key finding:** Rectangle boundaries remain consistent, confirming **Opening selectively removes noise while preserving structure.**

Closing Column (Figure 2)

Rectangle size remains consistent across all kernel sizes. Hole filling: 117 pixels (0.03%) at 3×3; 1,004 pixels (0.23%) at 9×9; 3,322 pixels (0.75%) at 15×15; 7,784 pixels (1.75%) at 30×30; 16,812 pixels (3.79%) at 50×50; 49,123 pixels (11.07%) at 80×80. **Key finding:** Rectangle boundaries remain consistent, confirming **Closing selectively fills holes while preserving structure.**

B.4.2 Cross-Operation Comparison

Comparing operations at the same kernel size (Figure 2):

Kernel Size	Erosion	Dilation	Opening	Closing
3×3	Slight shrinkage	Slight expansion	Rectangle preserved	Rectangle preserved
30×30	-80,673 pixels (-18.18%)	+152,216 pixels (+34.31%)	Rectangle preserved (-20,788, -4.69%, noise)	Rectangle preserved (+7,784, +1.75%, holes)

Finding: Opening and Closing preserve structure; Erosion and Dilation substantially alter it.

B.5 Qualitative Analysis: Zoomed Region Study

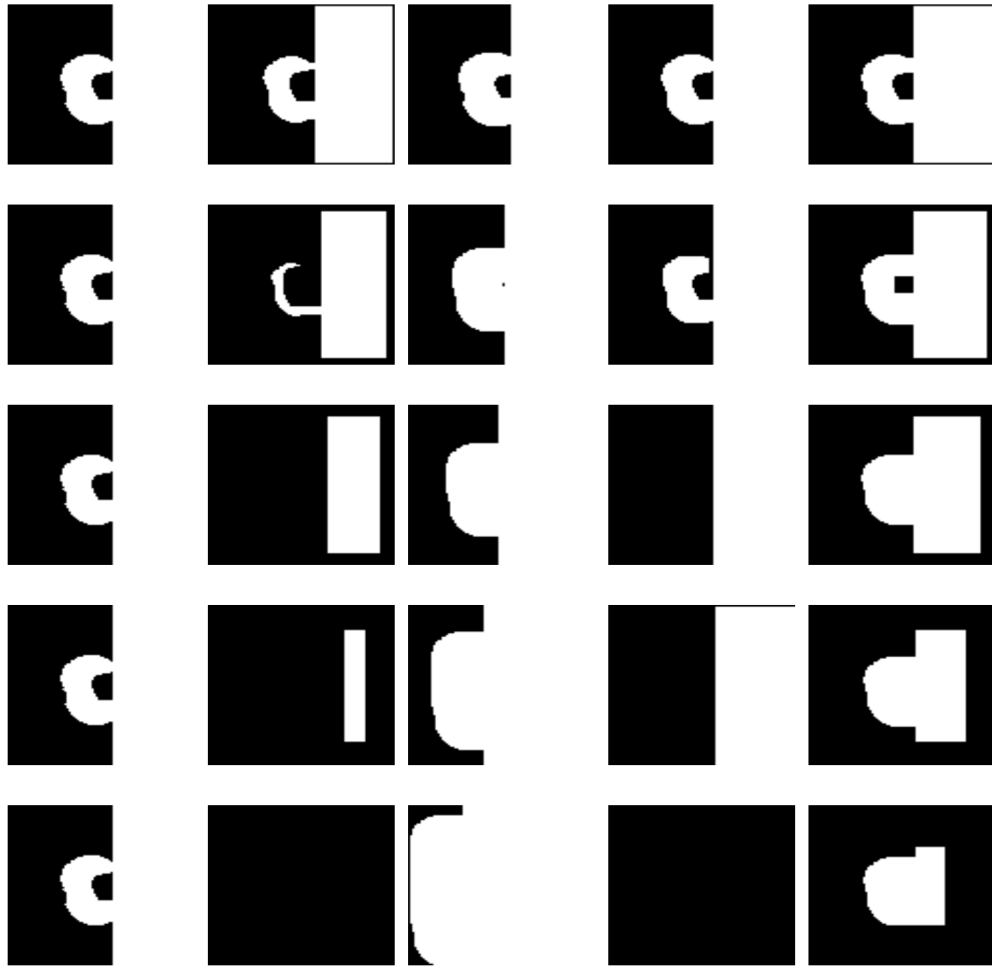


Figure 3: Zoomed region analysis showing detailed pixel-level effects of morphological operations on a selected white noise element.

We examine a selected region (coordinates 301-370, 727-803; $\sim 70 \times 76$ pixels) containing a medium-sized white noise element to observe pixel-level effects.

B.5.2 Visual Observations (Figure 3)

3×3: Erosion eliminates the blob; Opening removes it (erosion eliminates, dilation cannot restore); Dilation/Closing enlarge it.

9×9: Erosion removes blob with slight surrounding effect; Opening removes blob with larger footprint; Dilation/Closing significantly enlarge blob.

15×15: Erosion creates larger black region; Dilation dominates view; Opening shows larger footprint; Closing shows significant expansion.

30×30+: Effects extend far beyond original boundaries; Erosion creates extensive black regions; Dilation fills most of view; Opening/Closing affect entire region.

Key Finding: Kernel size directly correlates with processing footprint. Small kernels (3×3) precisely target elements; medium kernels (9×9, 15×15) affect element and surroundings; large kernels (30×30+) process areas far beyond target, confirming over-processing risk.

B.6 Discussion

B.6.1 Kernel Size Selection Strategy

Component analysis (Section B.3.3): white noise median = 442 pixels (range: 139-1,164,276); black holes median = 419 pixels (range: 1-423,090).

Empirical Selection Strategy:

Element Size	Recommended Kernel	Evidence	Visual Verification	Trade-off
Small (≤ 25 pixels)	3×3 or 5×5	3×3 Opening removes 258 pixels (0.06%)	Zoomed analysis confirms precise removal without affecting surroundings	High precision, limited scope
Medium (25-100 pixels)	9×9 or 15×15	9×9 removes 4,654 pixels (1.05%), 15×15 removes 13,674 pixels (3.08%)	Ablation study shows effective processing while preserving main structure	Balanced precision and coverage
Large (100-500 pixels)	30×30 or 50×50	30×30 removes 20,788 pixels (4.69%), 50×50 removes 23,708 pixels (5.34%)	Ablation study confirms processing of larger elements with some main structure impact	Good coverage, noticeable main structure impact
Very Large (> 500 pixels)	70×70 or 80×80	80×80 removes 23,708 pixels (5.34%), showing saturation	Ablation study shows potential over-processing, affecting main structure significantly	Maximum coverage, significant main structure impact

B.6.2 Operation Selection Guidelines

Operation	Visual Evidence	Quantitative Evidence	Application
Opening	Ablation study shows rectangle size preserved across all kernel sizes (443,388-419,938 white pixels vs 443,646 original)	Removes only 0.06%-5.34% of pixels (mostly noise) vs Erosion’s 2.01%-45.93%	Primary goal is noise removal while preserving main structure
Closing	Ablation study shows rectangle boundaries preserved across all kernel sizes (443,763-492,769 white pixels vs 443,646 original)	Adds only 0.03%-11.07% of pixels (mostly holes) vs Dilation’s 2.08%-113.87%	Primary goal is hole filling while preserving main structure
Erosion	Ablation study shows progressive rectangle shrinkage (from 443,646 to 239,897 white pixels with 80×80)	Removes 2.01%-45.93% of pixels, affecting both noise and main structure	Intentional structure shrinkage is desired
Dilation	Ablation study shows progressive rectangle expansion (from 443,646 to 948,810 white pixels with 80×80)	Adds 2.08%-113.87% of pixels, affecting both holes and main structure	Intentional structure expansion is desired

B.6.3 Combined Operations: Empirical Observations

Sequence	Visual Observation	Quantitative Check	Recommendation
Opening then Closing	Opening removes noise first (verified in ablation study), then Closing fills remaining holes	Opening removes 23,708 pixels, Closing adds up to 49,123 pixels—net addition suggests hole filling dominates	Preferable when noise removal is prioritized; creates cleaner structure for hole filling
Closing then Opening	Closing fills holes first (verified in ablation study), then Opening removes noise	Closing adds up to 49,123 pixels, Opening removes up to 23,708 pixels—net addition suggests hole filling effect persists	Viable when hole filling is prioritized; may enlarge noise before removal

Recommendation: Both sequences are viable. **Opening then Closing** is preferable when noise removal is prioritized, as it creates a cleaner structure for subsequent hole filling.

B.7 Conclusions

Kernel Size Impact

Erosion removes 2.01%-45.93% of pixels; Dilation adds 2.08%-113.87% as kernel size increases. Opening effect plateaus at 30×30 (23,708 pixels removed), indicating optimal range.

Operation Efficiency

Opening removes 0.06%-5.34% of pixels while preserving structure (443,388-419,938 vs 443,646 original). Closing adds 0.03%-11.07% while preserving boundaries (443,763-492,769 vs 443,646 original). Erosion and Dilation are non-selective, affecting 2.01%-45.93% and 2.08%-113.87% respectively.

Practical Recommendations

Optimal kernel range: **15×15 to 30×30** (balanced effect: 3.08%-4.69% removal for Opening, 0.75%-1.75% addition for Closing). These kernels preserve main structure while effectively processing random elements. Median element sizes (442 and 419 pixels) align with this range.

Problem Statement Interpretation

The statement suggests a kernel “just large enough to encompass any of the random elements.” While maximum elements exceed 1,000,000 pixels, median elements are ~400 pixels. Empirical results show 30×30 to 50×50 kernels provide optimal balance. **Conclusion:** Kernel size should match typical element size (median), not maximum, to avoid over-processing while maintaining effectiveness.

References

1. ITU-T Recommendation T.81 (1992). *Information technology – Digital compression and coding of continuous-tone still images – Requirements and guidelines*. (JPEG standard)
 2. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
 3. Witten, I. H., Neal, R. M., & Cleary, J. G. (1987). Arithmetic coding for data compression. *Communications of the ACM*, 30(6), 520-540.
 4. Serra, J. (1982). *Image Analysis and Mathematical Morphology*. Academic Press.
-

End of Report